

```
// Module descriptor file (*.gwt.xml)
<inherits name="com.google.gwt.logging.Logging"/>
// Property configuration in *.gwt.xml
<set-property name="gwt.logging.enabled" value="TRUE"/>
<set-property name="gwt.logging.consoleHandler" value="ENABLED"/>
// Handler for Root Logger
Logger rootLogger = Logger.getLogger("");
// Handler for Child of Root Logger
Logger childLogger = Logger.getLogger("childLogger");
```

Enabling or disabling logging, changing log levels and logging the handler configuration can be done programmatically in Java code, or by setting properties in the project module file (*.gwt.xml). All the log messages can be logged into the Development Shell of GWT, an IDE console, or the FireBug console, by configuring the corresponding logger handler.

JavaScript Native Interface (JSNI)

Eventually, we might need to integrate handwritten JavaScript functions or third-party JavaScript libraries/widgets. GWT allows developers to integrate these JavaScript libraries into the client side of applications through the JSNI interface. During GWT compilation, Java source code is converted into JavaScript, and that's when these JSNI interfaces are directly used in the final output. The concept of JSNI is inspired from Java native interfaces. The key features of GWT JSNI are its ability to implement a Java method as a JavaScript snippet, and the ability to call JavaScript from Java, and vice-versa. JSNI functions have to be declared natively and as static functions. All JavaScript code has to be embedded into comment blocks inside the function, as follows:

```
// JSNI function
public static native void alert(String msg) /*-{
    $wnd.alert(msg);
}*/;

// Calling JS function from Java function
public void onClick() {
    alert("calling JSNI Function");
}

//Sample invocation signature
[instance-name]&qualified-class-name::method-name(input-param-signature)(input-param-values)
```

In order to invoke Java code from a JSNI function, it has to use a detailed method signature approach to differentiate between overloaded functions.

Internationalisation

Due to the globalisation of IT applications, it has become mandatory to develop application support for specific countries and regions. The GWT framework provides a

separate module enabling you to develop one Web application that can display text in different languages. The GWT module for internationalisation is named *i18n*, and has to be inherited in the module descriptor file. The default locale used will be *locale=en_US*. Static string *i18n* is implemented by the *Constants/Messages* interface.

```
// Module descriptor file (*.gwt.xml)
<inherits name=" com.google.gwt.i18n.I18n" />

//Define I18NSamplesConstants.properties & I18NSamplesConstants_
el_fr.properties file
welcome=welcome           welcome= accueil
//Constants
welcome=welcome {0}       welcome= accueil {0}
// Messages

//Create an interface with same name as properties file
public interface I18NSamplesConstants extends Constants {
    String welcome();
}

//Dynamically create object for constants interface
I18NSamplesConstants myConstants = (I18NSamplesConstants) GWT.
create(I18NSamplesConstants.class);
gwtUIItem.setTitle(myConstants.welcome());
```

The *Constants* interface can be used to display a set of constant values of different types, for elements. The *Method* interface can be used to format and display values, along with parameters to accept input dynamically. We can define the type of locale to be loaded either in the module descriptor file, or dynamically in the application URL.

Dynamic string i18n is a little slower than a static approach, but can be used when integrating GWT applications with already existing server-side localisation systems. The module's host page can be configured to have the localised string to be displayed and these can be displayed appropriately.

HTML5 support in GWT

GWT supports the HTML5 canvas tag, using which you can build pages with dynamic image/audio/video support. Full support for HTML5 in classes of GWT is still under development, though a lot of open source projects have implemented this.

Local or client-side storage

HTML5 defines a standardised way to store large amounts of information at the client side. Earlier, you could store only 4 KB of information, with a maximum of 20 KB per domain, using cookies. Now, using HTML5 local storage, you can store a maximum of 5 MB of information, persistently cached at the client side. This reduces the number of hits to the server for small queries. GWT